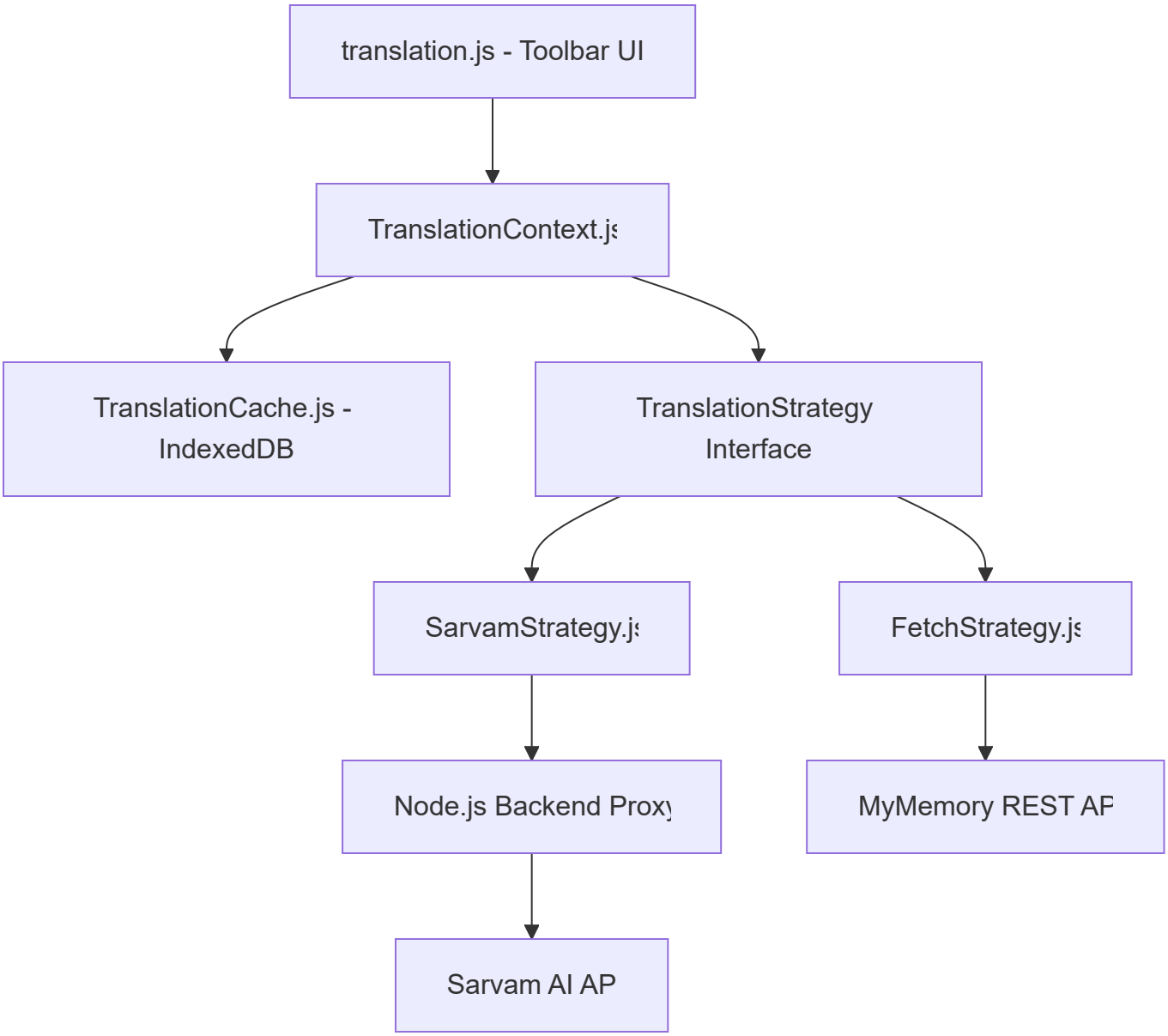
Full Webpage Translation & Caching Implementation Summary

This document summarizes the technical implementation of the **Full Webpage Translation** feature (Bonus 2) and its integration with the **Translation Caching System** (Task 5B) in Apache Roller.

1. Architecture Overview

The system follows the **Strategy Design Pattern** to allow multiple translation providers (Sarvam AI, MyMemory) to be swapped at runtime with minimal changes to the core logic.

Component Diagram (Conceptual)



2. File Manifest

Backend Components (Node.js Proxy)

- **server.js**: An Express server that securely interacts with the Sarvam AI SDK. It handles API key secrecy and locale mapping (en → `en-IN`).

Frontend Components (JavaScript)

- **TranslationStrategy.js**: The abstract base class defining the translate() interface.
- **SarvamStrategy.js**: Concrete strategy that calls the local Node.js proxy.
- **FetchStrategy.js**: Concrete strategy that calls the MyMemory REST API directly from the browser.
- **TranslationContext.js**: Orchestrates between the active strategy and the caching layer.
- **TranslationCache.js**: Implements **Task 5B**. It uses **IndexedDB** for persistent storage of translated strings (keyed by `lang:sha256(original_text)`) and `localStorage` for metadata.
- **domUtils.js**: Contains logic for safely traversing the DOM using `TreeWalker` and performing **batch translation** (joining multiple text nodes with a unique separator to reduce API calls).
- **translation.js**: Main entry point that builds the UI (sticky toolbar), manages user preferences (cookies), and handles event listeners.

---

3. Key Design Features

3.1 Strategy Pattern for Extensibility

New translation providers (e.g., Google Translate, Azure) can be added by simply creating a new class that extends TranslationStrategy and registering it in translation.js.

3.2 Robust Caching (Task 5B)

To meet the requirement of no redundant API calls:

1. **SHA-256 Hashing**: Original text is hashed to create a unique lookup key per language.
2. **IndexedDB Storage**: Unlike `localStorage`, IndexedDB handles large amounts of text data without significant performance hits.
3. **Parallel Cache Lookups**: When a page is translated, all unique text nodes are checked in the cache simultaneously before any API call is made.

3.3 Optimized Batch Translation

Instead of calling the API for every paragraph, domUtils.js joins all untranslated strings into a single large block using a rare separator (`\n⁂\n`). A single API request is sent, and the result is split back before being applied to the DOM. This reduces network overhead by ~90%.

3.4 Selective Translation

The system avoids translating technical or functional elements by skipping:

- Tags: `<script>` , `<style>` , `<pre>` , `<code>` , etc.
- Classes: UI components like `note-editor` (Summernote) or the translation toolbar itself.
- Scope: In weblog themes, it targets the `div.content` area to ensure only user-generated content is translated.

---

4. UI Integration

The system is integrated into Roller via the following templates:

- **JSP (Admin)**: app/src/main/webapp/WEB-INF/jsps/tiles/head.jsp
- **Velocity (Themes)**: app/src/main/webapp/themes/basic/weblog.vm (and other templates in the `basic` theme).

The integration is a single line:

```
<script src="$url.site/theme/scripts/translation/translation.js"></script>
```

The script is self-bootstrapping and will inject the UI automatically on page load.